$ curl /users/123 | usecase | repository | db

# Clean Architecture

// 分層，但不互相認識

| Controller | Usecase | Repository | Presenter |
| --- | --- | --- | --- |
| 服務生 | 主廚 | 倉管 | 打包員 |

$ cat blueprint.md

# 今日路線

**01 / 為什麼**
全黏在一起的程式怎麼讓你崩潰

**02 / 四個角色**
服務生、主廚、倉管、打包員

**03 / 實作**
TypeScript 完整範例 + 依賴注入

**04 / 驗收**
換 DB、換格式、加快取——各改哪層？

核心規則只有一條：你只跟上一層說話，下層的細節不關你的事。

PART 01

# 為什麼需要它？

先看一段讓人崩潰的程式

# 全黏在一起

```
// UI 直接連資料庫
app.get('/users', (req, res) => {
  const users = db.query('SELECT * FROM users');
  res.json(users.map(u => ({
    name: u.name,
    email: u.email,
    createdAt: u.created_at.format('YYYY-MM-DD')
  })));
});
```

INCOMING REQUESTS

老闆：「我們要換 GraphQL！」
→ 你崩潰（路由、格式全重寫）

老闆：「加個快取！」
→ 改 10 個地方

老闆：「換 MongoDB！」
→ SQL 散落各處，找都找不完

# 分層之後：改「怎麼做」，不動「做什麼」

**換資料庫？**
只改 Repository

**換 API 格式？**
只改 Presenter

**換前端框架？**
只改 Controller

INVARIANT

商業邏輯（Usecase）永遠不動 — 它是你的核心資產，活得比任何框架久。

PART 02

# 四個角色

一間餐廳就講完了

# 餐廳編制

**Controller = 服務生**
接單、把參數轉成程式能用的格式、傳話給廚房。不能寫商業邏輯！

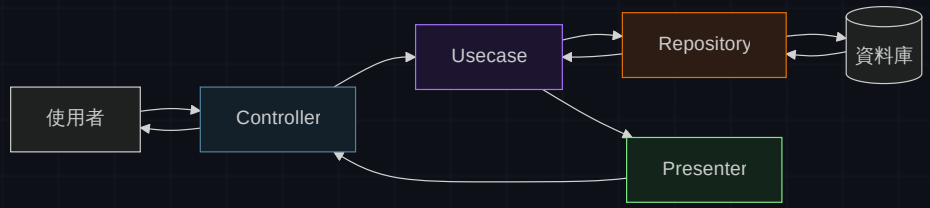
**Usecase = 主廚**
醃多久、火候多大——商業邏輯的家。不知道資料庫長怎樣、不知道前端要 JSON 還是 XML

**Repository = 倉管**
主廚喊「拿雞蛋」就去拿。從 MySQL、MongoDB 還是 API 拿？主廚不關心

**Presenter = 打包員**
JSON 盒子還是 XML 盒子？只管怎麼呈現，不管資料哪來

重點：廚房不知道菜市場在哪。大家只管自己的事。

# 請求流程：資料只往內流




**去程**
UI → Usecase → Repository → DB

**回程**
DB → Repository → Usecase → Presenter → UI，永遠不反向

PART 03

# TypeScript 實作

「取得使用者」完整走一遍

# Repository：先定契約

```ts
export interface UserRepository {
  getUser(id: number): Promise<User | null>;
}

// 實作：用 MySQL —— Usecase 完全不知道
export class MySQLUserRepository
  implements UserRepository {
  async getUser(id: number) {
    const rows = await db.query(
      'SELECT * FROM users WHERE id = ?', [id]);
    return rows[0] ?? null;
  }
}
```

# Usecase：商業邏輯的家

```ts
export class GetUserUseCase {
  constructor(
    private userRepository: UserRepository
  ) {}

  async execute(id: number) {
    const user =
      await this.userRepository.getUser(id);
    if (!user) return null;

    // 真正的商業邏輯：30 天未登入視為無效
    if (user.createdAt < thirtyDaysAgo()) {
      return null;
    }
    return user;
  }
}
```

**依賴介面，不依賴實作**

建構子注入 UserRepository —— MySQL 還是 Mongo？不知道也不需要知道

# Presenter 與 Controller

```
// Presenter：只管怎麼呈現
export class UserPresenter {
  static toResponse(user: User | null) {
    if (!user) return { error: 'User not found' };
    return {
      id: user.id,
      name: user.name,
      created_at: user.createdAt.toISOString(),
    };
  }
}
```

# 組裝端：依賴注入

```
// routes/user.ts —— 具體選擇集中在這
const repo = new MySQLUserRepository();
const usecase = new GetUserUseCase(repo);
const controller = new UserController(usecase);

router.get('/users/:id', controller.getUser);
```

COMPOSITION ROOT

Controller 不知道資料庫是什麼、Usecase 不知道前端要什麼 —— 每層都只跟介面說話。

PART 04

# 驗收時間

老闆的三個需求，各改哪一層？

# 三個需求，三次「只改一層」

**「換成 MongoDB！」**
新增 MongoDBUserRepository 實作介面，組裝端換一行 new。其他層零改動。

**「改輸出 XML！」**
只改 Presenter 的 toResponse()。其他層零改動。

**「查詢太慢，加快取！」**
CacheUserRepository 包住 MySQLUserRepository（裝飾器模式），先查快取再查 DB。Usecase 完全沒動。

$ 每個需求都只動一層 — 這就是分層的投資報酬率。

CHECKPOINT_01

使用者抱怨查詢太慢，你決定加快取（Cache）。該改哪一層？

A Usecase：在商業邏輯裡加一段查快取的程式

B Controller：收到請求先看看快取有沒有

C Repository：包一層 CacheUserRepository，先查快取再查 DB

# FAQ 快問快答

**這不就是 MVC 嗎？**

MVC 常把邏輯與存取黏在一起；Clean Architecture 是把「做什麼」和「怎麼做」分家。

**寫這麼多檔案不累嗎？**

寫 10 個檔案累，還是改 100 個地方時崩潰累？現在省 1 小時，未來用 100 小時還。

**小工具也要這樣？**

不用。但希望它活過 2 年的東西，就值得分層。

**Repository 要寫介面嗎？**

要。介面讓 Usecase 不知道你用什麼資料庫——明天才換得掉。

# 今日重點

**一條規則**
資料流只能往內，不能往外

**四個角色**
服務生、主廚、倉管、打包員——各管各的

**依賴介面**
具體選擇集中在組裝端（依賴注入）

**驗收標準**
換 DB、換格式、加快取——各只改一層

分層，但不互相認識 —— 商業邏輯永遠不動。

# Layers separated.

下一步：Spring Boot × DDD，看同樣的思想在後端怎麼落地

```
$ curl /users/123
controller → usecase → repository → presenter
```